

Andrew Yuan
ECE357 - Pset 4
Fall 2025

1a) Assuming ulimit is not set to 0 so that coredump files can be generated, one can create these coredump files via the kill system call. Using the pid of the process, we can kill it using `kill(pid_t pid, SIG)` when the program hangs where SIG is some signal that generates a core dump file. For example, we can open another terminal and then use `kill(pid_t pid, SIGQUIT)`. Without using the kill system call, we can also invoke SIGQUIT to terminate and generate a core dump process by using the associated key combination `Ctrl+\`. (I think this is what you what were referring to with the hint you gave me)

1b) We would want to map the BACKSPACE key to the SIGINT signal to terminate the current program just like pressing CTRL-C. We could use the stty command to do this mapping, doing `stty intr 8` or `^H`, where 8 is ASCII and ^H is the control code for backspace. This would allow us to torture some freshmen in ECE160.

1c) Take for example the scenario where one writes 65536 bytes into a pipe. Since the buffer can only contain 4096 bytes, not all of the bytes can be written at once (not atomic) and thus until we clear this buffer by reading, the process remains in the sleeping state. Since waiting for read/writing system calls are considered “long”, they are interruptible thus if a signal like SIGINT comes during this block, the signal wakes up the write syscall and returns early. This would appear as if the write system call completed successfully, even though only 4096 of the 65536 bytes were written through the pipe.

1d) If even signal 9 does not terminate the process, I would explain to my junior employee that he’s not getting a raise anytime soon and that the process is most likely in an uninterruptible sleep or hung. This means that signals, even SIGKILL, cannot terminate the process since in this waiting state, signals are not acted upon. For instance, some waits that are un-interruptable include waiting for disk I/O so this must be resolved.

1e) An interprocessor interrupt is an interrupt used on multiprocessor systems, where one CPU core needs to interrupt another CPU core. Although the state of the two CPUs may be different (i.e one returning from Kernel to User), instead of waiting an entire clock cycle for the target CPU to handle the signal, the kernel on the sending CPU can issue an interprocessor interrupt which immediately forces the target CPU to run the interrupt handler, which gives control back to the kernel where it can process the signal without any delay. Essentially interprocessor interrupts enable multiprocessor systems to adequately coordinate and deal with signals across different CPUs.

2a) Signal 15 is SIGTERM and since the count received is much less than the niter set, we can assume that SIGTERM is a non-real time signal and so these signals do not get queued by the kernel. If multiple of the same signal arrives before the signal handler can address it, they just collapse into one in the eyes of the handler.

```
• (base) tunabeluga@AndrewGaming:~/ECE357/ps4$ ./a.out 1000 15
Child got signal 2, count is 72
Parent got wait status 8000
• (base) tunabeluga@AndrewGaming:~/ECE357/ps4$ ./a.out 2000 15
Child got signal 2, count is 74
Parent got wait status 8000
• (base) tunabeluga@AndrewGaming:~/ECE357/ps4$ ./a.out 5000 15
Child got signal 2, count is 59
Parent got wait status 8000
```

2b) The parent gets a wait status of 8b which in Hex is 139 meaning a segfault occurred within the child. SA_NOMASK's behavior is "Do not automatically block signal while being handled", and since we are sending a large amount of signals in succession, the signal handler ends up being called re-entrantly. Each time the signal handler runs, it consumes stack space so we quickly overflow the stack and trigger a segmentation fault.

```
• (base) tunabeluga@AndrewGaming:~/ECE357/ps4$ ./a.out 50000 15
Parent got wait status 8b
```

2c) We no longer see the segfault issue that appeared in 2b. This is because changing the sigaction flags to 0 means the signal currently being handled is automatically blocked while its handler is executing. Any additional incoming signals are deferred until the current handler finishes, which prevents the recursive reentry issue seen in 2b.

```
• (base) tunabeluga@AndrewGaming:~/ECE357/ps4$ ./a.out 50000 15
Child got signal 2, count is 7786
Parent got wait status 8000
• (base) tunabeluga@AndrewGaming:~/ECE357/ps4$ ./a.out 500000 15
Child got signal 2, count is 79591
Parent got wait status 8000
• (base) tunabeluga@AndrewGaming:~/ECE357/ps4$ ./a.out 1000000 15
Child got signal 2, count is 163904
Parent got wait status 8000
```

2d) We see count be a much larger percentage of niter than before. In 2a, we see that count < 10% of niter, but now we see count be as large as almost 75% of niter. This is because signals 34-63 are real-time signals (32,33 are used by the threads library) and thus each signal is queued in the kernel until it is delivered. As a result, the child process receives and handles many more signals than before.

```
• (base) tunabeluga@AndrewGaming:~/ECE357/ps4$ kill -l
  1) SIGHUP      2) SIGINT      3) SIGQUIT     4) SIGILL      5) SIGTRAP
  6) SIGABRT     7) SIGBUS     8) SIGFPE      9) SIGKILL     10) SIGUSR1
 11) SIGSEGV    12) SIGUSR2    13) SIGPIPE    14) SIGALRM     15) SIGTERM
 16) SIGSTKFLT  17) SIGCHLD   18) SIGCONT    19) SIGSTOP     20) SIGTSTP
 21) SIGTTIN    22) SIGTTOU   23) SIGURG     24) SIGXCPU     25) SIGXFSZ
 26) SIGVTALRM  27) SIGPROF   28) SIGWINCH   29) SIGIO        30) SIGPWR
 31) SIGSYS     34) SIGRTMIN  35) SIGRTMIN+1 36) SIGRTMIN+2 37) SIGRTMIN+3
 38) SIGRTMIN+4 39) SIGRTMIN+5 40) SIGRTMIN+6 41) SIGRTMIN+7 42) SIGRTMIN+8
 43) SIGRTMIN+9 44) SIGRTMIN+10 45) SIGRTMIN+11 46) SIGRTMIN+12 47) SIGRTMIN+13
 48) SIGRTMIN+14 49) SIGRTMIN+15 50) SIGRTMAX-14 51) SIGRTMAX-13 52) SIGRTMAX-12
 53) SIGRTMAX-11 54) SIGRTMAX-10 55) SIGRTMAX-9  56) SIGRTMAX-8  57) SIGRTMAX-7
 58) SIGRTMAX-6  59) SIGRTMAX-5 60) SIGRTMAX-4  61) SIGRTMAX-3  62) SIGRTMAX-2
 63) SIGRTMAX-1  64) SIGRTMAX

• (base) tunabeluga@AndrewGaming:~/ECE357/ps4$ ./a.out 50000 34
Child got signal 2, count is 38260
Parent got wait status 8000

• (base) tunabeluga@AndrewGaming:~/ECE357/ps4$ ./a.out 100000 34
Child got signal 2, count is 45546
Parent got wait status 8000

• (base) tunabeluga@AndrewGaming:~/ECE357/ps4$ ./a.out 500000 34
Child got signal 2, count is 99319
Parent got wait status 8000

• (base) tunabeluga@AndrewGaming:~/ECE357/ps4$ ./a.out 500000 42
Child got signal 2, count is 100463
Parent got wait status 8000
```

Fun extra stuff: So for small values, we get count = niter. But for larger niters, even with real-time signals that they weren't equal. I was thinking these signals got "lost" but the queueing nature should counteract this, but then I figured a possible reason could be our queue limit. There should be a limit to this which I guess is RLIMIT_SIGPENDING.

```
• (base) tunabeluga@AndrewGaming:~/ECE357/ps4$ ./a.out 1000 59
Child got signal 2, count is 1000
Parent got wait status 8000

• (base) tunabeluga@AndrewGaming:~/ECE357/ps4$ ./a.out 10000 59
Child got signal 2, count is 10000
Parent got wait status 8000

• (base) tunabeluga@AndrewGaming:~/ECE357/ps4$ ./a.out 40000 59
Child got signal 2, count is 37772
Parent got wait status 8000
```